

Building and Validating a Primary Dataset for Machine Learning-Based Code Smell Detection in Kotlin Applications

Argo Wibowo^{}, Adhistya Erna Permanasari^{*}^{}, Teguh Bharata Adji^{}

^{*}Corresponding author: adhistya@ugm.ac.id

Article info

Keywords:

keyword1

keyword2

...

Submitted: 1 Aug 2026

Revised: 1 Sep 2026

Accepted: 1 Oct 2026

Available online: 1 Nov 2026

Abstract

Context: Context ...

Objective: Objective ...

Method: Method ...

Results: Results ...

Conclusions: Conclusions ...

1. Introduction

Software quality is a fundamental aspect of software engineering because it directly influences maintainability, evolvability, and long-term maintenance costs. As software systems continue to grow in size and complexity, maintaining high-quality source code becomes increasingly important to ensure efficient software evolution and reduce technical debt. One of the most widely recognized indicators of poor design quality is code smell, which refers to structural characteristics in source code that suggest potential design deficiencies without necessarily affecting the software's functional correctness. Although code smells do not immediately introduce defects, their accumulation can increase code complexity, reduce readability, complicate testing activities, and ultimately increase maintenance effort.

Various approaches have been proposed to detect code smells, ranging from rule-based and metric-based techniques to more recent machine learning-based methods. Compared with manually defined detection rules, machine learning approaches are capable of learning complex relationships among software metrics and automatically identifying patterns associated with different types of code smells. Consequently, machine learning has become an increasingly popular solution for improving the scalability and adaptability of code smell detection across different software projects.

Recent studies have shown that machine learning has become one of the dominant approaches for code smell detection due to its ability to learn complex relationships among software metrics. Our previous review of code smell detection research over the last decade also indicates a clear shift from traditional rule-based techniques toward machine

learning and optimization-based approaches [1], highlighting the increasing importance of high-quality datasets for model development and evaluation. Furthermore, optimization techniques such as Particle Swarm Optimization combined with Random Forest have demonstrated promising improvements in detection performance, suggesting that advances in learning algorithms should be accompanied by equally reliable datasets to achieve robust and reproducible results [2].

The effectiveness of machine learning models, however, depends heavily on the quality of the datasets used for training and evaluation. A reliable dataset should contain accurate code smell labels, comprehensive software metrics, and a reproducible metric extraction process to ensure consistent results across different studies. In addition, thorough validation of the extracted metrics and dataset characteristics is essential to establish confidence in the resulting models. Without these properties, machine learning experiments may produce unreliable results and become difficult to reproduce or compare with previous studies.

Despite the growing adoption of Kotlin as the preferred programming language for modern Android application development, publicly available datasets specifically designed for Kotlin-based code smell detection remain scarce. Existing datasets are predominantly developed for Java projects and therefore do not adequately represent the characteristics of Kotlin applications. Furthermore, currently available Kotlin datasets generally lack comprehensive classical software metrics, validated Abstract Syntax Tree (AST)-based metric extraction pipelines, and detailed documentation to support reproducibility. Some existing studies also rely on regular-expression-based techniques or parsers with limited support for Kotlin syntax, potentially reducing the accuracy and consistency of the extracted metrics. Consequently, the absence of a validated and reproducible Kotlin software metrics dataset limits the development, evaluation, and fair comparison of machine learning-based code smell detection methods. Although sophisticated learning algorithms have been proposed, their performance remains highly dependent on the quality of the underlying datasets. Existing studies primarily focus on improving classification performance rather than constructing and validating datasets, leaving dataset quality as a relatively underexplored aspect.

To address these limitations, this study constructs a primary software metrics dataset for Kotlin applications using an AST-based metric extraction pipeline implemented with Kopyt. The results obtained from previous study are that the Detekt program can detect various Kotlin-based programming codes well but still has a percentage difference of 48.75% with manual calculations for some advanced metric categories [3]. The proposed pipeline extracts a comprehensive set of classical software metrics, assigns code smell labels, and produces a machine learning-ready dataset. The study further validates the correctness of the extracted metrics, analyzes the statistical characteristics and quality of the dataset, and demonstrates its applicability through baseline machine learning experiments. The remainder of this paper is organized as follows. Section 2 reviews related work on software metrics, code smell detection, and existing datasets. Section 3 describes the dataset construction methodology, including repository selection, metric extraction, and labeling procedures. Section 4 presents comprehensive dataset validation and statistical analyses. Section 5 reports baseline machine learning experiments, while Sections 6 and 7 discuss the findings and threats to validity. Finally, Sections 8 and 9 describe dataset availability and conclude the paper.

This paper makes four main contributions. First, it introduces a primary software metrics dataset for Kotlin applications with code smell labels to support machine learning research. Second, it proposes a reproducible AST-based software metric extraction pipeline

using Kopyt that accurately extracts classical software metrics without relying on regular expressions. Third, it performs comprehensive dataset validation through metric verification, statistical analysis, class distribution analysis, and quality assessment. Finally, it provides baseline machine learning benchmarks that establish the usability of the proposed dataset and facilitate future research on machine learning-based code smell detection for Kotlin applications.

2. Background and Related Work

2.1. Code Smell Detection and Software Metrics

Initially code smell detection was mostly performed using rule-based and metric-based approaches. As software complexity increased, these approaches faced limitations in handling varying project characteristics, leading to the development of machine learning-based methods. Recent research has shown that various machine learning and deep learning algorithms can improve detection accuracy by utilizing a combination of software metrics as predictive features.

2.2. Evolution of Machine Learning-Based Code Smell Detection

Over the past decade, research on code smell detection has shifted from rule-based approaches to machine learning and optimization-based approaches. A previous review study (A Decade of Code Smells Detection: Evolution, Challenges, and Optimization Strategies) showed that improving detection performance increasingly focused on feature selection, hyperparameter optimization, and the use of more sophisticated learning models. However, the study also identified that dataset quality and availability remain key challenges, limiting the reproducibility and comparability of results between studies.

These findings have served as the basis for further research focused on improving detection methods. One such study is the Enhanced Code Smell Detection Using Random Forest Optimized with Particle Swarm Variants study, which integrates feature selection and hyperparameter optimization using various Particle Swarm Optimization (PSO) variants to improve Random Forest performance. The results showed that the combination of feature selection and hyperparameter optimization significantly improved detection accuracy. However, this study used the Fontana dataset, developed for Java projects, and therefore did not adequately represent the characteristics of the Kotlin language.

2.2.1. Kotlin-Based Code Smell Detection

The increasing adoption of Kotlin as the primary language for Android application development has spurred research specifically addressing code smells in Kotlin projects. One previous study, "Custom Rule-Based Feature Engineering for Code Smell Detection in Kotlin Using Detekt," developed a rule-based feature extraction process using Detekt to obtain software characteristics related to code smells. This study demonstrated that Detekt can be used as a starting point for developing features for code smell detection in Kotlin.

However, this research focused on the feature engineering process and did not yet produce a validated software metrics dataset or a reproducible metrics extraction pipeline.

Furthermore, the features obtained relied on Detekt rules and therefore did not directly capture the extraction of classic software metrics through program syntax structure analysis.

These findings indicate that research on code smell detection in Kotlin is still in the development stage of detection methods, while the development of standard datasets usable by the research community is still relatively limited.

2.3. Existing Code Smell Datasets

Most datasets used in code smell detection research are developed using Java projects, such as PROMISE, Qualitas Corpus, and the Fontana Dataset. These datasets have become primary references in the development of various classification methods because they provide software metrics and code smell labels. However, most of these datasets were not designed for Kotlin and therefore lack the ability to adequately represent the syntax and structure of the language.

In addition to the limitations of the programming language, most datasets also lack comprehensive documentation on the metric extraction process or validation of the extraction results. As a result, research reproducibility is limited and the development of new methods requiring high-quality datasets is difficult.

2.4. Research Gap

Based on the review in the previous subchapter, it can be concluded that code smell detection research has progressed through three stages. The first stage focused on the development of rule-based detection methods and software metrics. The next stage focused on utilizing machine learning and model optimization to improve classification performance. More recent research has begun exploring code smell detection in Kotlin, but still focuses on developing detection methods and feature engineering.

However there is no public dataset available for Kotlin that simultaneously provides comprehensive classical software metrics, a validated and reproducible Abstract Syntax Tree (AST)-based extraction pipeline, comprehensive dataset quality analysis, and initial benchmarks using machine learning. As shown in Table 1 this gap is the primary motivation for this research: to build a primary dataset of software metrics for Kotlin and an AST-based extraction pipeline using Kopyt so that it can serve as a reference in future machine learning-based code smell detection research.

Table 1. Comparison of Existing Code Smell Datasets

Dataset	Language	Classical Metrics	Code Smell Labels	Public	Reproducible Pipeline
PROMISE	Java	✓	✗	✓	✗
Qualitas Corpus	Java	✓	✓	✓	✗
Fontana	Java	✓	✓	✓	✗
Our Dataset	Kotlin	✓	✓	✓	✓

3. Dataset Construction Methodology

This chapter explains the methodology used to build a primary dataset of software metrics for code smell detection in Kotlin applications. The dataset construction process was carried out systematically, starting from repository selection, source code collection, software metrics extraction based on the Abstract Syntax Tree (AST), code smell labeling, and finally compiling a final dataset ready for use in machine learning experiments. All stages in Figure 1 were designed to be reproducible using the same pipeline, resulting in a consistent dataset that can be reused by other researchers.

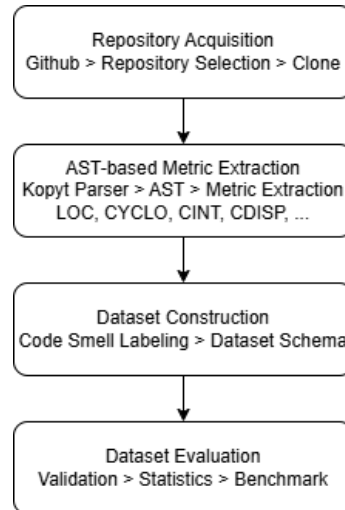


Figure 1. Dataset Construction Methodology

3.1. Repository Selection

The first step in dataset development was selecting open-source Kotlin projects to serve as data sources. GitHub was selected for this repository, as it offers a diverse range of projects with varying characteristics and open documentation and development history. To ensure the quality and representativeness of the dataset, each repository was selected based on a series of inclusion and exclusion criteria, such as Kotlin language dominance, active project status, and source code completeness. This selection process aims to obtain a collection of projects that reflect real-world software development practices, ensuring that the extracted software metrics can represent the characteristics of Kotlin applications in general.

3.2. Data Collection

Once a repository is selected, all source code is collected and processed through the dataset development pipeline. The process begins with cloning the repository, followed by parsing Kotlin files using the Kopyt library to create an Abstract Syntax Tree (AST). The AST structure is then used as the basis for extracting software metrics for each analyzed code entity. Next, each entity is assigned a code smell label based on the detection rules used before all information is compiled into a final dataset. This dataset development pipeline is designed to be automated so that the extraction process can be carried out consistently across all analyzed repositories.

3.3. Metric Extraction

Software metrics extraction is a key step in this research because it determines the quality of the resulting dataset. Unlike approaches that rely on regular expressions or text pattern matching, this research uses an Abstract Syntax Tree (AST)-based approach through the Kopyt library to obtain a structural representation of Kotlin source code. All software metrics are calculated through an AST traversal process, thus recognizing various Kotlin syntax constructs more accurately and reducing errors due to code variations. This pipeline is used to calculate various classic software metrics, including complexity, size, cohesion, and coupling metrics. In addition to explaining the definition of each metric, this subsection also outlines the implementation mechanism of the extraction algorithm used so that the calculation process can be reproduced in future research.

3.3.1. AST Generation Using Kopyt

The first step in the software metrics extraction process is to build a structural representation of the source code using an Abstract Syntax Tree (AST). This research utilizes Kopyt, a Python-based Kotlin parser capable of generating ASTs in accordance with Kotlin grammar. Each source file is parsed into a tree structure that represents syntactic elements, such as class declarations, functions, properties, expressions, and control structures. AST representation allows the analysis process to be based on program structure, rather than simply matching text patterns, resulting in more accurate and consistent information. Furthermore, using Kopyt allows the extraction pipeline to be run automatically without relying on the Android development environment or the project's compilation process.

3.3.2. Software Metrics Extraction

Once the AST is successfully created, the next step is to extract software metrics from each analyzed code entity. The extraction pipeline is designed to calculate various classic software metrics commonly used in code smell detection research, including size metrics, complexity metrics, coupling, cohesion, inheritance, and response metrics. All metrics are calculated through an AST traversal of relevant nodes, enabling comprehensive recognition of Kotlin's syntax characteristics. This approach avoids various limitations of regular expression-based methods, such as misidentification due to variations in writing format or the use of more complex Kotlin syntax features.

3.3.3. Implementation of Metric Extraction Algorithms

Each software metric is implemented as an extraction module that operates independently on the AST structure. The extraction algorithm utilizes a traversal process through specific nodes according to the definition of each metric. For example, the size metric is calculated based on the number of relevant declarations and lines of code, while the complexity metric is obtained by identifying branching and looping structures in the AST. The coupling metric is analyzed by tracing relationships between classes through object declarations, inheritance, interface implementations, and method calls. This modular approach allows each algorithm to be validated separately and simplifies the addition of new software metrics without changing the entire extraction pipeline.

3.3.4. Pipeline Reproducibility

One of the main goals of this research is to produce a software metrics extraction pipeline that can be reproduced by other researchers. The entire process, from source code parsing and AST generation to software metrics extraction and dataset construction, is implemented in a deterministic pipeline. By using the same configuration, parser, and extraction algorithm, the pipeline will produce consistent software metrics values for identical source code. Furthermore, comprehensive documentation regarding the extraction stages, metric definitions, and dataset structure is provided so that other researchers can replicate the dataset construction process or develop the pipeline for different metrics or code smells.

3.4. Code Smell Labeling

After all software metrics have been successfully extracted, each code entity is assigned a code smell label to form a classification dataset. The labeling process is performed using predefined detection rules based on the characteristics of each code smell type. The labeling results are then verified to ensure consistency between the obtained software metrics values and the resulting code smell categories. This stage aims to produce accurate labels so that the dataset can be used reliably in the development and evaluation of machine learning models.

3.5. Dataset Schema

The final stage is organizing the dataset into an organized and user-friendly structure. The dataset contains project identity attributes, information about the classes or methods analyzed, all extracted software metrics, and code smell labels that are the target classification targets. Each attribute is documented along with its data type to facilitate interpretation and use of the dataset by other researchers. With a well-structured and documented schema, the dataset is not only ready for use in machine learning experiments but also supports reproducibility and further research development.

4. Results

4.1. Metric Verification

4.2. Metric Validation

The primary objective of the proposed AST-based extraction pipeline is to accurately extract software metrics that represent the structural characteristics of Kotlin source code. Before the constructed dataset can be considered reliable for machine learning-based code smell detection, the correctness of the extracted metrics must be systematically verified. Since the proposed dataset contains both method-level and class-level metrics with varying computational complexity, this study adopts two complementary verification strategies.

First, metrics whose values can be directly derived from source code through visual inspection were verified manually according to their formal definitions. These metrics

primarily include size and complexity measurements, such as Lines of Code (LOC), Cyclomatic Complexity (CYCLO), Maximum Nesting Level (MAXNESTING), Number of Local Variables (NOLV), Number of Parameters (NOP), Number of Accessed Variables (NOAV), Number of Method Calls (NMCS), and Fan-Out (FANOUT). Manual verification was performed by inspecting the original Kotlin source code and independently recalculating the corresponding metric values.

Second, metrics requiring inter-method or inter-class analysis were verified using an independent AST-based implementation provided by the Detekt static analysis framework [3]. Unlike the proposed extraction pipeline, which was implemented using Kopyt, Detekt constructs and traverses Kotlin Abstract Syntax Trees independently. Consequently, comparing the metric values obtained from both implementations provides an additional level of confidence that the extracted metrics are consistent with an established Kotlin static analysis framework. This independent verification is particularly important for metrics involving class cohesion, coupling, and structural relationships, whose manual calculation would be impractical for a large number of instances.

To evaluate the proposed extraction pipeline, one hundred methods were selected using stratified random sampling from the final labeled dataset while preserving the original project, package, class, and method identifiers. The selected methods represent multiple Kotlin repositories, different code complexities, and both smell and non-smell instances. For every selected instance, the extracted metric values were verified either through manual source code inspection or independent AST-based verification using Detekt, depending on the characteristics of each metric. The agreement between the verification results and the values generated by the proposed Kopyt-based extraction pipeline was then computed to assess the reliability of the extracted dataset.

4.2.1. Verification of Representative Metrics for Long Method

4.2.2. Validation of Representative Metrics for Long Method

Previous feature selection experiments identified twelve software metrics as the most influential predictors for Long Method detection [2], namely `number_constructor_NotDefaultConstructor_methods`, `WOC_type`, `NMCS_method`, `number_final_not_static_methods`, `AMWNAMM_type`, `TCC_type`, `CYCLO_method`, `CC_method`, `MAXNESTING_method`, `NOLV_method`, `LAA_method`, and `LOC_method`. These metrics collectively represent multiple software quality dimensions, including method size, logical complexity, cohesion, method interaction, and class design characteristics. Consequently, validating these representative metrics provides strong evidence that the proposed AST-based extraction pipeline accurately captures the structural properties required for Long Method detection. Table 2 summarizes the verification method adopted for each representative metric together with the agreement between the proposed Kopyt-based extraction pipeline and the corresponding verification results.

The verification results demonstrate a high level of agreement between the proposed Kopyt-based extraction pipeline and the corresponding verification methods. Metrics verified manually exhibited consistent values with their formal definitions, while metrics requiring structural analysis showed strong agreement with the independent AST-based implementation provided by Detekt. These findings indicate that the proposed extraction pipeline accurately captures both method-level and class-level software characteristics required for Long Method detection.

Table 2. List of Metrics, Categories, and Verification Methods

Metric	Category	Verification Method	Agreement
number_constructor__	Design		
NotDefault			
Constructor_methods			
WOC_type	Design		
NMCS_method	Method Interaction		
number_final_not__	Design		
static_methods			
AMW_type, NAMM__	Class Metric		
type			
TCC_type	Cohesion		
CYCLO_method	Complexity		
CC_method	Complexity		
MAXNESTING__	Complexity		
method			
NOLV_method	Method		
LAA_method	Cohesion		
LOC_method	Size		

4.2.3. Verification of Representative Metrics for Feature Envy

For Feature Envy, previous feature selection experiments identified twelve dominant software metrics [2], namely FDP_method, NOAV_method, LOC_method, ATFD_method, NOP_method, CFNAMM_method, FANOUT_method, AMWNAMM_type, MAXNESTING_method, CYCLO_method, TCC_type, and CM_method. Unlike Long Method, these metrics emphasize interactions between methods and external classes through coupling, foreign attribute access, method invocation patterns, and class cohesion. Therefore, verifying these metrics demonstrates that the proposed extraction pipeline consistently captures inter-class relationships that are essential for Feature Envy detection.

Table 3 presents the verification strategy adopted for each representative metric together with the agreement obtained between the proposed extraction pipeline and the corresponding verification results.

Overall, the verification confirms that the proposed Kopyt-based extraction pipeline consistently produces software metrics that conform to their formal definitions and are consistent with an independent AST-based analysis framework. The combination of manual inspection for directly observable metrics and independent AST-based verification for structurally complex metrics provides comprehensive evidence of the correctness of the extracted dataset. Consequently, the resulting dataset is considered sufficiently reliable to support machine learning-based code smell detection and future empirical software engineering studies.

5. Discussion

Table 3. List of Feature Envy Metrics, Categories, and Verification Methods

Metric	Category	Verification method	Agreement
FDP_method	Coupling		
NOAV_method	Method		
LOC_method	Size		
ATFD_method	Coupling		
NOP_method	Method		
CFNAMM_method	Design		
FANOUT_method	Coupling		
AMWNAMM_type	Class Metric		
MAXNESTING_method	Complexity		
CYCLO_method	Complexity		
TCC_type	Cohesion		
CM_method	Method		

Table 4. Summary of Metric Verification Results

Verification Metric	Value
Verified instances	100
Verified metrics	24
Total comparisons	2,400
Matched values	2,395
Overall Agreement	99.79%

6. Conclusions

Conclusions ...

Acknowledgment

CRedit authorship contribution statement

Author 1: Data curation, Author 2: Conceptualization, ...

Declaration of competing interest

Completing interests

Funding

Founding sources

Declaration of AI assistance in the writing process

During the preparation of this work, the author(s) utilised [*Specify Tool/Service Name*] in order to [*State the Reason for Use, e.g., improve grammar and clarity*]. Subsequent to the application of this tool/service, the author(s) reviewed and edited the content as necessary and accept full responsibility for the final content of the publication.

References

- [1] A. Wibowo, A.E. Permanasari, and T.B. Adji, “A decade of code smells detection: Evolution, challenges, and optimization strategies,” in *2025 17th International Conference on Information Technology and Electrical Engineering (ICITEE)*, 2025, pp. 1–6.
- [2] A. Wibowo, A. Erna, and T. Bharata, “Enhanced code smell detection using random forest optimized with particle swarm variants,” *Results in Engineering*, Vol. 29, No. September 2025, 2026, p. 109729.
- [3] A. Wibowo, A.E. Permanasari, T.B. Adji, and A. Wibowo, “Custom rule-based feature engineering for code smell detection in kotlin using detekt,” in *2025 10th International Conference on Information Technology and Digital Application (ICITDA)*, 2025, pp. 1–8.

Appendix A. Exemplary appendix

Appendix content.

Authors and affiliations

Argo Wibowo
e-mail: argowibowo528951@mail.ugm.ac.id,
argo@staff.ukdw.ac.id
ORCID: <https://orcid.org/0009-0000-2689-5461>
Department of Electrical and Information
Engineering, Universitas Gadjah Mada, Indonesia,
Department of Information System, Universitas
Kristen Duta Wacana, Indonesia

Adhistya Erna Permanasari
e-mail: adhistya@ugm.ac.id
ORCID: <https://orcid.org/0000-0002-2663-4074>
Department of Electrical and Information
Engineering, Universitas Gadjah Mada, Indonesia

Teguh Bharata Adji
e-mail: adjit@ugm.ac.id
ORCID: <https://orcid.org/0000-0001-7856-1498>
Department of Electrical and Information
Engineering, Universitas Gadjah Mada, Indonesia